

项目验收报告

项目架构&各组件功能&软件栈

架构

各组件功能

- 1. API Server：使用**Gin**框架，与etcd交互增删查改，其他组件通过HTTP请求向API Server同步/获取信息，而API Server通过RabbitMQ消息队列通知其他组件进行增删改查等操作。
- 2. Kubelet（继承Kube-Proxy）：管理容器运行时，创建删除Pod，进行Pod的状态更新和重启，进行Service本地路由规则的更新。
- 3. Controller：管理Replicaset、节点扩缩容等。
- 4. Scheduler：负责选择Pod调度到哪一个节点，当前使用随机算法进行选择。
- 5. NameServer：Go撰写的DNS服务器以及HTTP反向代理服务器。
- 6. KubeCtl：MiniK8s的CLI命令行工具，用于执行MiniK8s的各种操作。

分工和贡献度

名称	分工	贡献度
廖承凡	API Server, NameServer, Security Context	33.34%
刘世琪	Kubelet, Pod抽象和容错, Service, PV/PVC	33.34%
王理想	ReplicaSet, Serverless, auto-scaling, GPU, Kubectl, 资源监视 (cadvisor)	33.32%

项目详细信息

项目仓库地址

https://github.com/Hydroiodic/MiniK8S_SE3356_2024-2025-2

分支介绍

- main：主分支
- feat/kubectl：与MiniK8s CLI工具相关的功能实现

feat/serverless: Serverless相关功能的实现

feat/pv-pvc: PV/PVC相关功能的实现

feat/dns: DNS和反向代理相关功能的实现

注：由于分支是根据功能创建的，每一个新功能会新创建一个分支，大部分分支在合并进入main分支之后就删除了，这里仅展示一部分还未删除的分支。如果需要所有功能实现的历史记录，请通过Pull Requests记录查看。

CI/CD介绍

1. 触发条件

- `push`：当代码推送到 `main` 分支时，触发CI/CD流程。
- `pull_request`：当向 `main` 分支发起拉取请求时，触发CI/CD流程。

2. 构建作业（`build`）

- 运行环境：在 `ubuntu-22.04` 操作系统上运行。
- 步骤解析：
 - a. 构建项目：运行 `go build -v ./...` 命令，构建项目。
 - b. 设置Docker环境：使用 `docker/setup-buildx-action@v2` 设置Docker环境，为后续可能的Docker相关操作做准备。
 - c. 设置网络：创建一个Docker网络，配置子网和网桥名称，用于测试容器间通信。
 - d. 设置IPVS：运行 `bash scripts/ipvs/ipvs.sh` 脚本，用于测试Service。
 - e. 测试并生成覆盖率报告：运行 `go test -v -coverprofile=coverage.out ./test/...` 命令进行测试，并生成覆盖率报告 `coverage.out`。然后使用 `go tool cover` 将覆盖率报告转换为HTML格式。
 - f. 上传覆盖率报告：使用 `actions/upload-artifact@v4` 将覆盖率报告 `coverage.html` 上传为工作流的工件，便于后续查看。

3. 代码质量检查作业（`golangci-lint`）

- a. 对于不符合Lint的代码拒绝合并，统一团队的代码风格。

软件测试方法介绍

单元测试：

使用 `go tool cover` 检查路径覆盖率。

自底向上集成测试

使用Driver，模拟上层对下层的调用。

- 测试IMAGE服务能否正常拉取镜像，解析镜像路径等。
- 测试Container能否正常创建、启动、停止 Docker 容器。
- 测试Pod能否正常创建启动停止，其中各个Container能否正常启动，Container之间能否通过localhost互相通信。

新功能开发流程介绍

从main切出新分支，进行开发，使用PR合并到main。若冲突，将origin/main合并到该分支，处理冲突，提交后重新进行PR。

各个功能点

Node

Node同Kubelet，当Kubelet启动的时候发送注册请求给API Server，然后由API Server相应地在etcd数据库中添加一条记录。在我们的项目中，Node和Kubelet并未进行严格的区分。

Pod

使用方法

代码块

```
1 kubectl apply -f pod.yaml
2 kubectl delete pod nginx
3 kubectl get pods
```

Runtime

使用**Docker**作为容器运行时

利用Go API 与之通信

构建了Image、Container、Pod的三层抽象

使用Label为Pod中的Container标记Pod的参数，以便崩溃恢复时恢复Pod状态。

容器支持的功能大多可以使用Docker的HostConfig实现。

Pod抽象

首先创建Pause容器，新加入的容器指定共享其网络命名空间和IPC命名空间。

Pause容器拥有 `IsPause=true` 和Pod的Namespace和Name构成的Label `podNs + "_" + podName`，Kubelet通过扫描Pause容器个数即可知道本机运行的Pod数目。

Pause 容器的 HostConfig

使用Flannel网桥，并使用minik8s提供的DNS

HostConfig

```
1 // 让容器共享 IPC 资源，允许使用共享内存、信号量等机制
2 hostConfig := &container.HostConfig{
3     IpcMode:      container.IPCModeShareable,
4     NetworkMode:  container.NetworkMode("flannel"),
5     DNS:          []string{object.DNSClusterIP, "8.8.8.8"},
6 }
```

其他容器

代码块

```
1 // 普通容器在创建时会通过 Docker 的
2 // --net=container:<pauseId>、--ipc=container:<pauseId> 和 --pid=container:
  // <pauseId> 选项，
3 // 加入 Pause Container 的命名空间
4 hostConfig := &container.HostConfig{
5     NetworkMode: container.NetworkMode(pauseNsArg),
6     IpcMode:      container.IpcMode(pauseNsArg),
7     PidMode:      container.PidMode(pauseNsArg),
8     Mounts:       binds, // Handle VolumeMounts here.
9     GroupAdd:     combinedSecurityContexts.SupplementalGroups,
10 }
```

而Pod内其他容器则也打上相应标签，便于进行归类。

Docker 中 Container命名遵循 `podNs + "_" + podName + "_" + containerName`

Pod的几种状态

`GetPodStatus` 通过查询 Pause 容器和业务容器的状态，判断并返回 Pod 当前的状态。

1. 检查 Pause 容器是否存在

- 如果找不到 Pause 容器，说明 Pod 还没创建好，返回 `PodCreating`。

2. 遍历业务容器，检查每个容器的状态

- 如果有业务容器未创建，返回 `PodCreating`
- 如果有业务容器状态未知（获取失败），返回 `PodUnknown`

3. 收集所有业务容器的状态

- 统计每个容器的状态（Running、Created、Exited等）

4. 根据容器状态判断 Pod 状态

- 只要有一个容器 **Running**：Pod 状态为 `PodRunning`
- 所有容器都是 **Created**：Pod 状态为 `PodCreating`
- 其他情况：Pod 状态为 `PodCreating`。

Pod状态的自动恢复

`AutoRestartPod` 的主要逻辑如下

1. 检查 Pause 容器

- 通过 label 查找对应 Pod 的 Pause 容器。
- 如果找不到 Pause 容器，说明 Pod 已经丢失或未创建，直接执行：
 - 删除 Pod（`DeletePod`）
 - 重新创建 Pod（`CreatePod`）
 - 启动 Pod（`StartPod`）
 - 返回错误提示 “no pause container found...”

2. 检查业务容器

- 用 label 查找所有业务容器。
- 遍历 Pod 规范中的每个容器，尝试获取其状态。
- 如果有容器未创建或被删除，执行：
 - 删除 Pod
 - 重新创建 Pod
 - 启动 Pod
 - 返回错误提示 “container ... is not created yet...”

3. 检查并修复 Pod IP

- 如果 Pod 的 IP 为空，尝试通过 Pause 容器重新获取并写入。

4. 根据重启策略重启容器

- 获取 Pod 的重启策略（`Always`、`OnFailure`、`Never`）。
- 遍历所有业务容器的状态：
 - **Always/空**：只要容器是 dead、exited、created 状态，就 stop+start 重启。
 - **OnFailure**：如果容器 dead/exited 且 exitCode 非0，则 stop+start 重启。

- **Never**: 如果容器是 created 状态，则 start 一次。

Kubelet组件

代码块

```
1 go s.podController.Run(stopCh)
2 go s.serviceController.Run(stopCh)
3 go s.statusController.HeartBeatRoutine(stopCh)
4 go s.statusController.UpdatePodRoutine(stopCh)
```

创建四个协程，分别负责

- 处理Pod的添加和删除消息
- 处理Service的添加和删除消息
- 定时上报Kubelet状态（包括Pod状态和服务状态）
- 统计Pod状态，处理Pod重启

跨主机通信

在etcd中填写Flannel配置

代码块

```
1 etcdctl put /coreos.com/network/config '{"Network": "101.6.0.0/16",
  "SubnetLen": 24, "SubnetMin": "101.6.11.0", "SubnetMax": "101.6.19.0",
  "Backend": {"Type": "vxlan"}}'
```

使用**Flannel**的VXLAN模式，创建了新的Docker Bridge网络，名称为 `flannel`。

代码块

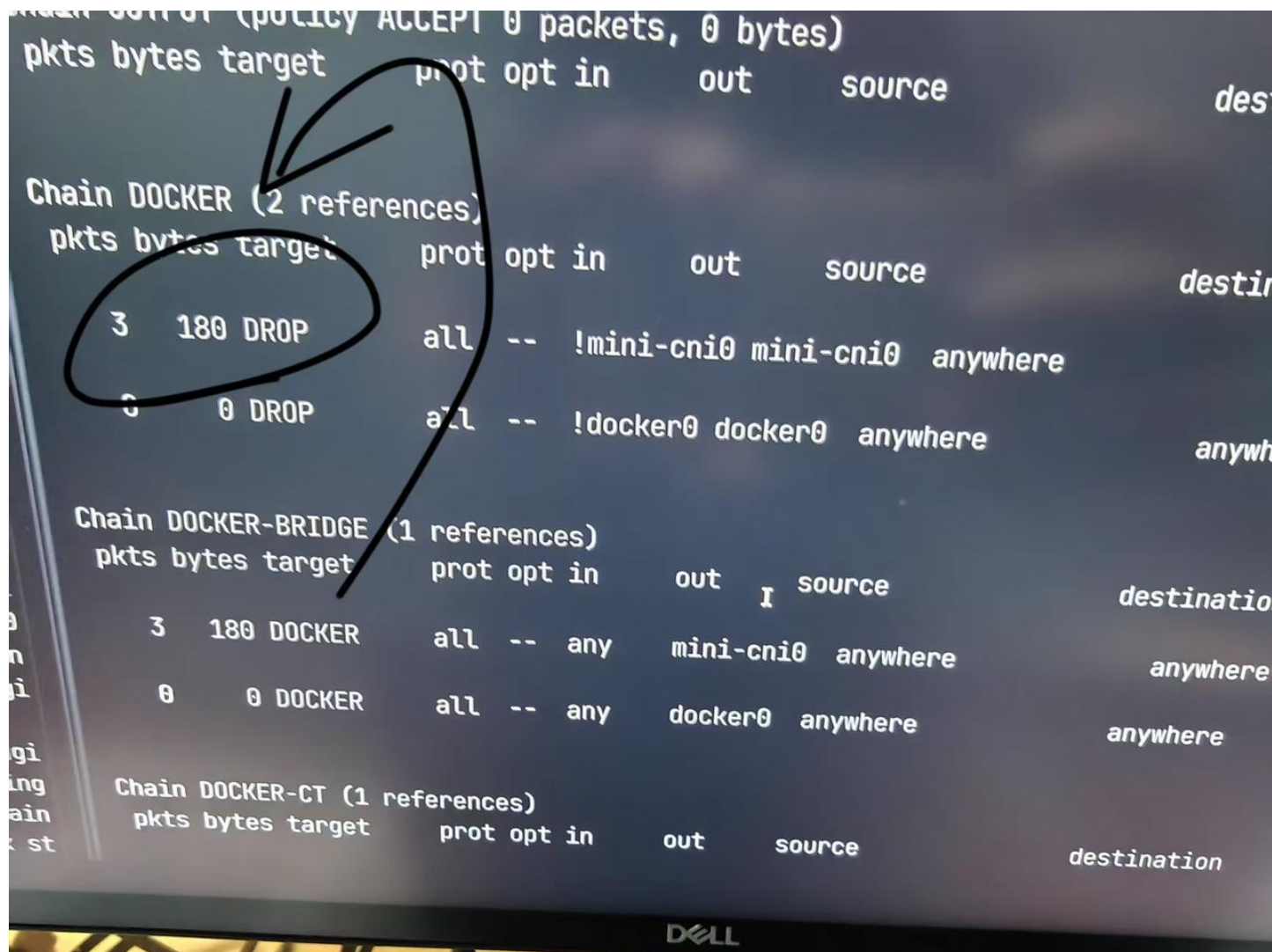
```
1 # 创建新的网络: flannel
2 # 网桥名称为mini-cni0
3 docker network create --attachable=true --subnet=${FLANNEL_SUBNET} -o
  "com.docker.network.driver.mtu"=${FLANNEL_MTU} -o
  "com.docker.network.bridge.name"="mini-cni0" flannel # 为docker创建一个flannel网
  络，它使用了这个flannel分配的子网；让这个网络在主机上的设备名称为mini-cni0
```

为了便于调试，通过配置`iptables`修改原有Docker的iptables配置，实现了宿主机到PodIP的访问。

代码块

```
1 sudo iptables -I FORWARD -j FLANNEL-FWD
```

过去Docker有一条IPTABLES规则，会自动DROP来自网桥外部的流量。



同步

API Server通过RabbitMQ消息队列通知Pod进行创建和删除，Kubelet进行相应处理。

Kubelet定时获取本地运行的Pod状态，并主动尝试恢复，然后定时向API Server汇报。

Service

功能由 `kubeproxy` 组件承担，集成在 `kubelet` 中，随之在各个节点启动，主要功能是配置IPVS进行负载均衡。

IPVS

主要使用IPVS实现：

创建一个名为 `mini-kube-ipvs0` 的网卡，用于绑定Virtual Service 的 IP，同时使用 `modprobe` 启用各种内核模块，允许多种负载均衡策略，如 `rr`，`lc`，`wlc`。

Service的同步

API Server 在 Service 创建时根据其Selector，与etcd中Pod的Label进行匹配，并将需要创建、更新、删除的Service发往kube-proxy，这一过程一定频率反复进行。

Cluster IP的实现

ClusterIP的分配由API Server决定，IP地址为 `222.111.x.x`，其中保留 `222.111.0.1` 作为DNS Service的地址，`222.111.0.2` 作为HTTP反向代理服务器的地址。

kube-proxy接到通知后，根据API Server分配的Cluster IP创建Virtual Service端点，根据匹配到的EndPoint创建IPVS的Real Server，进而实现Cluster IP访问。其中对TCP和UDP服务均进行了映射，也支持相应端口的配置。

对于NodePort 类型，IPVS中添加一条本机 `IP:NodePort` 到 `CLUSTER IP:Port` 的映射。

ReplicaSet

与MiniK8S系统的集成

ReplicaSet控制器在MiniK8S架构中作为控制平面的一部分：

1. 与API服务器交互：
 - 获取ReplicaSet和Pod信息
 - 更新ReplicaSet状态
 - 创建/删除Pod
2. 与调度器协作：
 - 依赖调度器(随机算法)来放置新创建的Pod
3. 与Kubelet配合：
 - 由各节点的kubelet管理创建的Pod
4. 与服务集成：
 - ReplicaSet创建的Pod可以通过Service暴露
 - IPVS负责这些Pod的负载均衡

核心组件分析

ReplicaSetController结构体

代码块

```
1 type ReplicasetController struct {}
```

控制器启动机制

代码块

```
1 func (rsc *ReplicasetController) Start() {
2     ticker := time.NewTicker(5 * time.Second) // 每5秒触发一次
3     defer ticker.Stop()
4
5     for range ticker.C {
6         rsc.CheckAllReplicaset()
7     }
8 }
```

- 每5秒执行一次全量ReplicaSet检查
- 使用Go的time.Ticker实现定时执行

核心功能实现

Pod创建功能:

代码块

```
1 func (rsc *ReplicasetController) CreatePod(num int, rs object.ReplicaSet) {
2     ci := apiserver.NewAPIClient("")
3
4     for range num {
5         var pod object.Pod
6         pod.Metadata.Labels = rs.Metadata.Labels
7         pod.Metadata = rs.Spec.Template.Metadata
8         pod.Metadata.Name = "Replica" + "-" + rsc.NewUid()[:5]
9         pod.Spec = rs.Spec.Template.Spec
10        pod.Status = object.PodStatus{
11            Phase:      "Running",
12            StartTime: time.Now(),
13        }
14        pod.Kind = "Pod"
15
16        err := ci.CreatePod(&pod)
17        if err != nil {
18            fmt.Println(err)
19        }
20    }
21 }
```

- 创建指定数量的Pod副本
- 为每个Pod生成带随机后缀的唯一名称

- 从ReplicaSet模板复制配置
- 初始化Pod状态为“Running”

Pod删除功能:

代码块

```
1 func (rsc *ReplicasetController) DeletePod(pods []object.Pod, num int) {
2     ci := apiserver.NewAPIClient("")
3     for i := range num {
4         err := ci.DeletePod(&pods[i])
5         if err != nil {
6             fmt.Println(err)
7             continue
8         }
9     }
10 }
```

- 从提供的Pod列表中删除指定数量的Pod
- 实现了基本的错误处理

ReplicaSet检查机制:

代码块

```
1 func (rsc *ReplicasetController) CheckAllReplicaset() {
2     ci := apiserver.NewAPIClient("")
3
4     replicaset, err := ci.GetReplicaset()
5     pods, err := ci.GetPods()
6
7     for _, rs := range replicaset {
8         curLabels := rs.Spec.Template.Metadata.Labels
9         var matchPods []object.Pod
10
11         // 查找匹配的Pod
12         for _, p := range pods {
13             if apiserver.HasMatchingLabels(curLabels, p.Metadata.Labels) {
14                 matchPods = append(matchPods, p)
15             }
16         }
17
18         rs.Status.AvailableReplicas = len(matchPods)
19
20         if len(matchPods) == rs.Spec.Replicas {
21             // 副本数量正确
```

```

22         rs.Status.AvailableReplicas = rs.Spec.Replicas
23         return
24     } else if len(matchPods) < rs.Spec.Replicas {
25         // 需要扩容
26         rsc.CreatePod(rs.Spec.Replicas-len(matchPods), rs)
27         rs.Status.AvailableReplicas = rs.Spec.Replicas
28     } else {
29         // 需要缩容
30         rsc.DeletePod(matchPods, len(matchPods)-rs.Spec.Replicas)
31         rs.Status.AvailableReplicas = rs.Spec.Replicas
32     }
33
34     err = ci.UpdateReplicaset(&rs)
35 }
36 }

```

- 从API服务器获取所有ReplicaSet和Pod信息
- 对每个ReplicaSet:
 - 通过标签查找匹配的Pod
 - 比较当前副本数与期望副本数
 - 自动扩缩容以匹配期望状态
 - 更新ReplicaSet状态

Auto Scaling

架构设计

HPA控制器采用主从式协程架构设计。主控制循环每15秒执行一次全量检查，负责HPA对象的生命周期管理。每个HPA对象有独立的监控协程，每5秒执行一次指标采集和扩缩容决策。

核心数据结构包含四个关键组件：

1. HpasMap - 存储当前管理的HPA对象缓存
2. Tickers - 维护每个HPA的检查定时器
3. QuitChs - 提供协程退出控制通道
4. API客户端 - 负责与Kubernetes API Server交互

核心工作流程

控制器初始化

启动过程首先初始化容器服务接口，随后启动cAdvisor监控组件。初始化完成后创建主检查循环，以15秒为周期执行全量检查。API客户端用于后续所有与集群的交互。

HPA全量同步

全量同步首先从API Server获取当前所有HPA对象。对于新增的HPA，会创建专属的检查协程和定时器。对于已删除的HPA，则清理相关资源并终止对应协程。

每个HPA协程启动后立即执行首次检查，之后按5秒间隔定期运行。协程通过退出通道实现优雅终止。

单HPA检查流程

代码块

```
1 func (hpaC *HPAController) CheckOneHPA(hpa object.HorizontalPodAutoscaler) {
2     // 获取所有pod对象
3     pods, err := hpaC.Ci.GetPods()
4     if err != nil {
5         fmt.Println(err)
6         return
7     }
8     // 根据hpa的配置，获得replicaSet
9     replicaset, err := hpaC.Ci.GetReplicasetName(
10         hpa.Spec.ScaleTargetRef.Name,
11     )
12     if err != nil {
13         fmt.Println(err)
14         // return
15         fmt.Println("该hpa已被删除")
16     }
17
18     // 现在已经获取到了replicaset，接下来需要获取到replicaset下的所有pod
19     var matchPods []object.Pod
20
21     for _, p := range pods {
22         if apiserver.HasMatchingLabels(
23             replicaset.Metadata.Labels,
24             p.Metadata.Labels,
25         ) {
26             matchPods = append(matchPods, p)
27         }
28     }
29
30     // // 计算replicaset级别的所有pod的资源利用情况，只是做一个简单的算术平均
31     podsMetricsEntries := make(map[string]object.PodMetrics, 0)
32
33     for _, pod := range matchPods {
34         containers := pod.Spec.Containers
```

```

35
36     var cpuUsage = 0.0
37
38     var memoryUsage = 0.0
39
40     for _, container := range containers {
41         fmt.Println(
42             pod.Metadata.Namespace + "_" + pod.Metadata.Name + "_" +
container.Name,
43         )
44
45         containercpuUsage, containermemoryUsage, err :=
cadvisorutils.GetContainerCPUandMem(
46             "localhost",
47             "8090",
48             pod.Metadata.Namespace+"_"+pod.Metadata.Name+"_"+container.Name,
49         )
50
51         if err != nil {
52             fmt.Println(err)
53             continue
54         }
55
56         cpuUsage += containercpuUsage
57         memoryUsage += containermemoryUsage
58     }
59
60     fmt.Printf("cpu利用率 %f\n", cpuUsage)
61     fmt.Printf("内存利用率 %f\n", memoryUsage)
62
63     podsMetricsEntries[pod.Metadata.Namespace+pod.Metadata.Name] =
64         object.PodMetrics{
65             Resources: map[string]float64{
66                 "cpu":    cpuUsage,
67                 "memory": memoryUsage,
68             },
69         }
70     }
71
72     rsMetricsResult := CalculateReplicaMetrics(
73         &hpa,
74         podsMetricsEntries,
75     )
76
77     // 当前replicaset的副本数, 应该取spec静态期望值, 而非真实值, 可以留给
replicaSetController一些调整时间

```

```

78     currentReplicaNum := replicaset.Spec.Replicas
79     // 期望副本数
80     desiredReplicaNum := CalculateDesiredReplicas(
81         &hpa,
82         currentReplicaNum,
83         rsMetricsResult,
84     )
85
86     fmt.Printf("当前replicaset的数量: %d\n", currentReplicaNum)
87
88     fmt.Printf("期望replicaset的数量: %d\n", desiredReplicaNum)
89     // 比较
90     if currentReplicaNum == desiredReplicaNum {
91         // 如果已经相等, 则不管
92         return
93     } else {
94         if currentReplicaNum < desiredReplicaNum {
95             // 如果当前副本数小于期望副本数, 则副本数自增1
96             replicaset.Spec.Replicas = currentReplicaNum + 1
97         } else {
98             replicaset.Spec.Replicas = currentReplicaNum - 1
99         }
100
101         // 发给api-server, 为这个replicaSet应用新的副本数; 注意这里直接使用create方
            法, 传递相同的静态配置 (不需要管UID的问题, 重建一份也没事), 只是副本数不同
102         err = hpaC.Ci.UpdateReplicaset(&replicaset)
103         if err != nil {
104             fmt.Printf("update replicaset failed : %s\n", err)
105         }
106     }
107 }

```

单次检查包含六个关键步骤:

1. 获取关联Pod和ReplicaSet
2. 筛选匹配标签的Pod
3. 通过cAdvisor采集容器指标
4. 计算ReplicaSet级别指标平均值
5. 根据算法确定期望副本数
6. 执行渐进式扩缩容调整

关键算法实现

指标聚合算法

指标聚合采用平均值计算策略。首先遍历HPA配置的所有指标类型，然后聚合所有Pod的对应指标值，最后计算算术平均值作为ReplicaSet级别指标。

副本数计算算法

副本计算采用最大值策略。对每个指标独立计算期望副本数，取所有结果中的最大值。计算时考虑当前副本数与指标使用率的比例关系，并确保结果在配置的最小最大副本数范围内。

容错机制

组件级容错

控制器对cAdvisor和API Server的异常进行捕获和处理。当组件不可用时记录错误但继续运行，避免单点故障导致整体不可用。

数据一致性保障

通过定期全量同步和状态重建机制保证数据一致性。控制器重启后能够重新建立完整的监控体系。

渐进式调整策略

采用每次±1的副本调整方式，避免指标波动引发剧烈扩缩容。这种设计提高了系统的稳定性。

与系统集成

与API Server集成

通过API客户端获取集群状态和提交变更。所有扩缩容决策都通过API Server持久化。

与cAdvisor集成

通过容器服务接口获取实时监控数据，为决策提供依据。默认使用8090端口进行数据采集。

与ReplicaSet控制器协同

通过修改ReplicaSet的副本数声明来实现扩缩容，两者协同工作确保期望状态与实际状态一致。

DNS

DNS分为两个部分：

1. 第一部分即字面意义的DNS，接收UDP域名解析请求，并返回对应的IP解析。
2. 第二部分是反向代理服务器，由于我们需要实现同一域名中不同路径路由到不同的Service，而单一的DNS并无法实现这个功能，因此引入反向代理服务器，将对应的路径路由到对应的Service。

两部分均使用Go语言实现，DNS部分调用 github.com/miekg/dns 实现，而反向代理服务器通过内置 `net/http` 库实现。它们本身作为内置Service复用Service机制，在API Server启动的时候自动添加，使用固定的Cluster IP——Pod启动的时候传入DNS的Cluster IP，这样Pod内部就可以使用DNS服务了。

配置文件示例：

代码块

```
1  kind: DNS
2  metadata:
3    name: my-dns
4    namespace: default
5    labels:
6      app: myapp
7  spec:
8    host: example.com
9    paths:
10     - path: /web
11       serviceName: web-service
12       servicePort: 8080
13     - path: /
14       serviceName: nginx-service
15       servicePort: 80
```

这将会在web-service有效的时候将访问前缀为 `/web` 的请求转发至其中，而如果web-service无效，则所有请求均被转发到nginx-service下。如果两个service均无效，则该DNS规则不生效。

容错

数据面容错

Kubelet可以通过本机运行的容器的Tag自动恢复出其上运行的Pod，在重启后继续定时向API Server汇报。

API Server在一定时长内没有收到Node的心跳，判断节点超时，之后从etcd中删除所有其上的Pod，之后的Pod部署便不会继续在該Node上进行。

代码块

```
1  // Only once: restore local pods status.
2  localPods, err := s.podController.podService.ListPods()
3  if err != nil {
4      log.Printf("Failed to fetch pods: %v", err)
5  }
6
7  // 清理 KubeProxy 本地状态
8  s.serviceController.IpvsOps.Clear()
9  s.serviceController.IpvsOps.Init()
10
11  log.Printf("Restoring local pods: %v", utils.ExtractPodNames(localPods))
```



```
12 s.kubelet.Mu.Lock()
13 s.kubelet.Pods = localPods
14 s.kubelet.Mu.Unlock()
```

ListPods 通过遍历所有 Pause 容器，结合业务容器和 label 信息，动态还原出当前节点上所有 Pod 的详细信息和状态。

1. 获取所有 Pause 容器

- 通过 label (`IsPauseLabelKey: "true"`) 查找当前节点上所有 Pause 容器。
- 每个 Pause 容器代表一个 Pod 的“网络/命名空间沙箱”。

2. 遍历每个 Pause 容器，组装 Pod 信息

- 对每个 Pause 容器：
 - 解析出所属 Pod 的 Namespace 和 Name (通过 label) 。
 - 获取 Pause 容器的 IP 作为 Pod IP。
 - 查找该 Pod 下所有业务容器 (通过 label, `IsPauseLabelKey: "false"`) 。
 - 去除 Pause 容器 label 中的 `IsPauseLabelKey` (只保留业务 label) 。

3. 构造 Pod 对象

- 用上面收集到的信息 (Metadata、Spec、Status) 组装成 `object.Pod` 结构体。
- 填充 Pause 容器 ID、业务容器列表、Pod IP、Label 等。

4. 返回所有 Pod 列表

- 返回当前节点上所有正在运行的 Pod 列表 (包含状态字段) 。

Serverless

架构设计

整体架构设计

Serverless模块采用分层解耦架构，包含三个核心层级：

1. 管理层 (Function Controller)

- 负责函数镜像构建、存储和管理
- 与Kubernetes API交互创建底层资源
- 使用缓存机制减少API调用

2. 事件层 (Event Trigger)

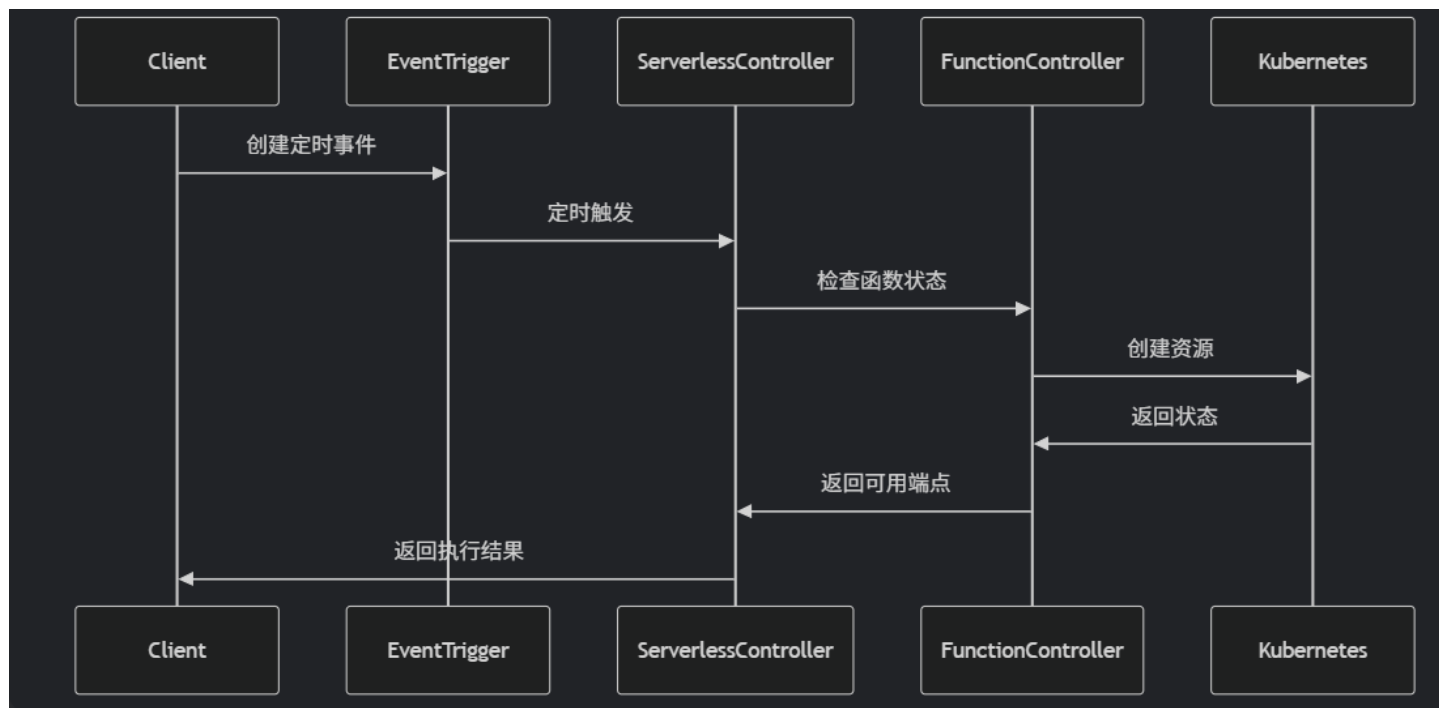
- 基于cron的时间调度引擎

- 支持秒级精度的定时触发
- 事件与目标的多对多映射

3. 服务层（Serverless Controller）

- 提供RESTful API网关
- 实现请求路由和负载均衡
- 内置自动扩缩容引擎

核心组件交互



Function Controller 深度解析

核心功能实现细节

函数创建流程：

1. 文件准备阶段

- 在 `/assets/allfunctions/<namespace>/<name>` 创建专属目录
- 写入用户上传的ZIP包（权限0777）
- 使用archiver库解压到目标目录

2. 镜像构建阶段

- 动态生成Dockerfile（包含基础镜像、代码拷贝等指令）
- 创建tar格式的构建上下文
- 调用Docker SDK构建镜像
- 使用Base64编码的认证信息推送镜像

3. 资源部署阶段

- 创建ReplicaSet（初始副本数为0）
- 配置Volume挂载用户代码
- 创建ClusterIP类型的Service
- 暴露10000端口供内部访问

目录结构规范

代码块

```
1  assets/allfunctions/
2  └── {namespace}/
3      └── {function-name}/
4          ├── function.zip          # 原始上传文件
5          ├── function.tar          # Docker构建上下文
6          ├── Dockerfile            # 自动生成
7          ├── {user-code}/          # 解压后的代码
8              ├── main.py           # 用户代码示例
9              └── requirements.txt  # 依赖文件
10 └── ...                           # 其他用户文件
```

Event Trigger 深度解析

定时事件引擎实现

事件处理流程：

1. 解析事件目标（Function/Workflow）
2. 验证目标可用性
3. 转换JSON数据为Map
4. 构造HTTP POST请求
5. 设置Content-Type头
6. 通过http.Client发送请求

事件状态管理

使用三层缓存结构：

1. EventMap：存储事件最新配置
2. Crons：维护活跃的定时器
3. QuitChs：控制协程生命周期

同步机制：

- 每15秒全量同步一次
- 对比API Server与本地缓存
- 增量更新（新增/删除/修改）

Serverless Controller 分析

核心功能

- API网关
- 自动扩缩容
- 流量统计
- Workflow编排

自动扩缩容策略

- 扩容算法：基于最近访问次数的平方根
- 缩容条件：5分钟无访问缩容至0
- 冷却时间：15秒内不重复扩缩容

Workflow引擎特性

1. 支持函数节点和条件节点
2. 参数自动传递
3. 条件分支判断

关键数据结构

Function Controller

代码块

```
1 type FunctionController struct {
2     cache map[string]object.Function // 函数缓存
3     ci     *apiserver.APIClient // API客户端
4 }
```

Event Trigger

代码块

```
1 type EventTriggerController struct {
```

```

2      EventMap map[string]*object.Event // 事件缓存
3      Crons    map[string]*cron.Cron   // 定时器
4      QuitChs  map[string]chan struct{} // 退出通道
5      Ci       *apiserver.APIClient   // API客户端
6  }

```

Serverless Controller

代码块

```

1  type Serverless_controller struct {
2      route_map      map[string]string // 路由表
3      timeVisitedQueue map[string][]time.Time // 访问时间队列
4      funcPodNums     map[string]int      // 函数Pod数量
5      // ...其他字段
6  }

```

典型工作流程

函数调用流程

1. 请求到达API网关
2. 检查路由表获取服务地址
3. 触发自动扩容(如果需要)
4. 转发请求到目标Pod
5. 返回响应

Workflow执行流程

1. 解析起始节点
2. 执行函数节点
3. 评估条件节点
4. 跳转到下一节点
5. 返回最终结果

持久化存储

PVC会查找当前所有可用且满足储存容量要求的PV，选择其中有效的进行绑定；若找不到，创建NFS类型的PV。

Pod通过PVC名称与PVC绑定，进而访问对应的PV，遵循如下规则：

1. 对于HostPath类型PV,在容器启动时直接挂载对应路径即可；
2. 对于NFS类型的PV,使用mount命令将NFS的IP和路径绑定到Pod专属的Volume目录，随后在启动时挂载该专属目录。

HostPath PV 支持

- 如果 PVC 绑定的 PV 是 HostPath 类型（`pv.Spec.HostPath != nil`），
- 调用 `handleHostPathVolume`，在宿主机上创建对应目录，并在 Pod 卷目录下创建软链接指向 HostPath 路径。
- 这样容器内访问挂载点时，实际访问的是宿主机的本地目录。

代码块

```
1  else if pv.Spec.HostPath != nil && pv.Spec.HostPath.Path != "" {
2      // 处理 HostPath 类型的 PV
3      err = vm.handleHostPathVolume(pv.Spec.HostPath.Path, volumePath)
4      ...
5  }
```

NFS PV 支持

- 如果 PVC 绑定的 PV 是 NFS 类型（`pv.Spec.NFS != nil`），
- 先确保挂载点目录存在，然后调用 `mountNFSVolume`，用 `mount -t nfs` 命令把远程 NFS 目录挂载到 Pod 卷目录下。
- 容器内访问挂载点时，实际访问的是远程 NFS 服务器上的共享目录。

代码块

```
1  if pv.Spec.NFS != nil && pv.Spec.NFS.Server != "" && pv.Spec.NFS.Path != "" {
2      ...
3      err = vm.mountNFSVolume(pv, volumePath)
4      ...
5  }
```

- **HostPath PV**：通过软链接把宿主机目录暴露给 Pod。
- **NFS PV**：通过系统 mount 命令把远程 NFS 目录挂载到 Pod 卷目录。
- 这两种 PV 都可以通过 PVC 绑定后被 Pod 使用，`MountVolumes` 会自动识别并处理。

Security Context

经过相当一部分资料查阅，安全上下文在Docker API中并没有现成的接口。经过测试，发现Docker中 `RunAsUser`、`RunAsGroup`、`GroupAdd` 具有下面的性质：

1. 如果 `RunAsUser` 不指定，则默认为镜像里指定的User运行。
2. 如果 `RunAsGroup` 不指定，则以 `RunAsUser` 在 `/etc/passwd` 中的用户组运行，且会带上 `/etc/group` 中的附加用户组；而如果指定，在 `GroupAdd` 为空的情况下，附加用户组仅会包含 `RunAsGroup` 的用户组。
3. 无论前面的配置如何，`GroupAdd` 只会以Merge的方式添加附加用户组。

基于此行为，以如下规则设定上面的变量：

1. 如果 `RunAsUser` 未指定，则指定为 `root`；否则为指定的用户。
2. 如果 `RunAsGroup` 未指定，则指定为 `root`；否则为指定的用户。
3. 如果指定模式为Strict，则设定 `GroupAdd` 的内容为 `SupplementalGroup` 和 `FsGroup` 的并集，因为指定 `RunAsGroup` 的情况下不会有任何默认附加组。
4. 如果指定模式为Merge，则从Docker Daemon中读取镜像中的 `/etc/group` 文件，找到当前 `RunAsUser` 的所有用户组 `Groups`，然后设定 `GroupAdd` 的内容为 `Groups`、`SupplementalGroup` 和 `FsGroup` 三者的并集。

最后有关 `FsGroup` 的设定，只需要对于挂载卷，递归遍历其下所有文件，修改所有用户组为 `FsGroup` 指定的内容，然后对挂载的目录设置SetGid权限即可，之后其下创建的所有文件均会归 `FsGroup` 用户组所有。

GPU

架构设计

整体架构

GPU Job控制器采用定时轮询机制，每10秒检查一次集群中的GPU Job状态变化。控制器主要负责处理GPU密集型作业的完整生命周期，包括作业文件处理、Docker镜像构建、镜像推送和Pod创建。

核心数据结构

代码块

```
1 type JobController struct {
2     cache map[string]object.Job // 缓存已处理的Job对象
3     ci     *apiserver.APIClient // Kubernetes API客户端
4 }
```

核心工作流程

控制器初始化

控制器启动时会创建API客户端并初始化缓存，然后启动一个定时器，每10秒执行一次全量检查。

作业检查循环

每次检查会获取集群中所有GPU Job，并与缓存比较，对新发现的Job触发创建流程。

作业创建流程

完整的作业创建包含以下步骤：

1. 创建作业目录结构
2. 解压用户上传的作业文件
3. 生成SLURM作业脚本
4. 构建Dockerfile
5. 打包构建上下文
6. 构建Docker镜像
7. 推送镜像到仓库
8. 创建运行Pod

关键功能实现

文件处理

- 在 `jobs/<namespace>/<job-name>` 目录下管理作业文件
- 支持ZIP格式的作业文件上传和解压
- 自动生成SLURM脚本文件

镜像构建

- 基于基础镜像构建作业专属镜像
- 通过Docker API进行镜像构建
- 自动设置环境变量包括输出文件、错误文件等

集群集成

- 与SLURM集群集成，支持GPU资源调度
- 与kubelet集成，创建运行Pod
- 与Docker Registry集成，推送构建的镜像

目录结构设计

代码块

```
1  jobs/
2  └─ <namespace>/
3      └─ <job-name>/
4          └─ job.zip          # 用户上传的原始文件
5          └─ job.tar          # Docker构建上下文
6          └─ Dockerfile       # 生成的Dockerfile
7          └─ <job>.slurm       # 生成的SLURM脚本
8          └─ <解压内容>       # 用户作业文件(如.cu文件)
```